Read Relationships

Now that we know how to connect data using **relationship Attributes**, let's see how to get and read the objects from a relationship.

Select a Hero

First, add a function `select_heroes()` where we get a hero to start working with, and add that function to the `main()` function:

Python 3.10+

```
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        result = session.exec(statement)
        hero_spider_boy = result.one()

# Code here omitted 👈

def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```python
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        result = session.exec(statement)
        hero_spider_boy = result.one()

        statement = select(Team).where(Team.id == hero_spider_boy.team_id)
        result = session.exec(statement)
        team = result.first()
        print("Spider-Boy's team:", team)

        print("Spider-Boy's team again:", hero_spider_boy.team)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Select the Related Team - Old Way

Now that we have a hero, we can get the team this hero belongs to.

With what we have learned **up to now**, we could use a `select()` statement, then execute it with `session.exec()`, and then get the `.first()` result, for example:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        result = session.exec(statement)
        hero_spider_boy = result.one()

        statement = select(Team).where(Team.id == hero_spider_boy.team_id)
        result = session.exec(statement)
        team = result.first()
        print("Spider-Boy's team:", team)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        result = session.exec(statement)
        hero_spider_boy = result.one()

        statement = select(Team).where(Team.id == hero_spider_boy.team_id)
        result = session.exec(statement)
        team = result.first()
        print("Spider-Boy's team:", team)

        print("Spider-Boy's team again:", hero_spider_boy.team)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Get Relationship Team - New Way

But now that we have the **relationship attributes**, we can just access them, and **SQLModel** (actually SQLAlchemy) will go and fetch the corresponding data from the database, and make it available in the attribute. ✨

So, the highlighted block above, has the same results as the block below:

Python 3.10+

```
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        result = session.exec(statement)
        hero_spider_boy = result.one()

# Code here omitted 👈

        print("Spider-Boy's team again:", hero_spider_boy.team)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```python
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        result = session.exec(statement)
        hero_spider_boy = result.one()

        statement = select(Team).where(Team.id == hero_spider_boy.team_id)
        result = session.exec(statement)
        team = result.first()
        print("Spider-Boy's team:", team)

        print("Spider-Boy's team again:", hero_spider_boy.team)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Tip

The automatic data fetching will work as long as the starting object (in this case the `Hero`) is associated with an **open** session.

For example, here, **inside** a `with` block with a `Session` object.

Get a List of Relationship Objects

And the same way, when we are working on the **many** side of the **one-to-many** relationship, we can get a list of the related objects just by accessing the relationship attribute:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Preventers")
        result = session.exec(statement)
        team_preventers = result.one()
```

```
        print("Preventers heroes:", team_preventers.heroes)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")
    team: Team | None = Relationship(back_populates="heroes")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team=team_z_force
        )
        hero_rusty_man = Hero(
            name="Rusty-Man", secret_name="Tommy Sharp", age=48, team=team_preventers
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)

        hero_spider_boy.team = team_preventers
        session.add(hero_spider_boy)
        session.commit()
        session.refresh(hero_spider_boy)
        print("Updated hero:", hero_spider_boy)

        hero_black_lion = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
        hero_sure_e = Hero(name="Princess Sure-E", secret_name="Sure-E")
        team_wakaland = Team(
            name="Wakaland",
            headquarters="Wakaland Capital City",
            heroes=[hero_black_lion, hero_sure_e],
        )
        session.add(team_wakaland)
```

```python
        session.commit()
        session.refresh(team_wakaland)
        print("Team Wakaland:", team_wakaland)

        hero_tarantula = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
        hero_dr_weird = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
        hero_cap = Hero(
            name="Captain North America", secret_name="Esteban Rogelios", age=93
        )

        team_preventers.heroes.append(hero_tarantula)
        team_preventers.heroes.append(hero_dr_weird)
        team_preventers.heroes.append(hero_cap)
        session.add(team_preventers)
        session.commit()
        session.refresh(hero_tarantula)
        session.refresh(hero_dr_weird)
        session.refresh(hero_cap)
        print("Preventers new hero:", hero_tarantula)
        print("Preventers new hero:", hero_dr_weird)
        print("Preventers new hero:", hero_cap)


def select_heroes():
    with Session(engine) as session:
        statement = select(Team).where(Team.name == "Preventers")
        result = session.exec(statement)
        team_preventers = result.one()

        print("Preventers heroes:", team_preventers.heroes)


def update_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Spider-Boy")
        result = session.exec(statement)
        hero_spider_boy = result.one()

        hero_spider_boy.team = None
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_spider_boy)
        print("Spider-Boy without team:", hero_spider_boy)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

That would print a list with all the heroes in the Preventers team:

```bash
```

Recap

With **relationship attributes** you can use the power of common Python objects to easily access related data from the database. 😎